

Smart History Tutor: Sistema Multiagente con Simulación Metaheurística

Bruno Jesús Pire Ricardo

Repositorio GitHub:

<https://github.com/Br1Pire/smart-history-tutor/tree/main>

Abstract. Se desarrolló un sistema denominado Smart History Tutor con el objetivo de implementar un tutor inteligente de Historia Universal basado en una arquitectura multiagente alineada con el modelo Retrieve-Augmented Generation (RAG). La versión extendida integra un módulo de simulación metaheurística que permite optimizar planes de clases mediante algoritmos genéticos y heurísticos. El sistema incorpora agentes especializados para recuperación semántica mediante FAISS, generación de respuestas con la API de Google Gemini, y simulación de estudiantes que modelan procesos de aprendizaje, motivación y retención de información usando lógica difusa. Los resultados demuestran la viabilidad de la solución propuesta para generar respuestas educativas de calidad y optimizar planes docentes en función del aprendizaje simulado de los estudiantes.

Keywords: RAG, multiagente, metaheurística, simulación, FAISS, sistemas difusos, aprendizaje, Historia Universal

1 Introducción

El proyecto Smart History Tutor se concibió con el propósito de diseñar un sistema inteligente capaz de responder preguntas y asistir en el aprendizaje de Historia Universal. Su desarrollo integró conocimientos adquiridos en Inteligencia Artificial, Simulación y Sistemas de Recuperación de Información, priorizando la creación de un entorno educativo automatizado que brinde soporte a estudiantes de enseñanza media superior. Inicialmente, la solución implementada se enfocó en la construcción de un tutor inteligente con arquitectura multiagente alineada con el modelo Retrieve-Augmented Generation (RAG), combinando técnicas de recuperación semántica y generación de texto. Posteriormente, se amplió el alcance del sistema mediante la incorporación de un módulo de simulación metaheurística, el cual permite optimizar planes de clases utilizando algoritmos genéticos y evaluaciones simuladas sobre agentes estudiantes modelados con procesos de aprendizaje, motivación y retención de información. Este enfoque integral propone no solo una herramienta de respuesta, sino también un sistema capaz de planificar y validar estrategias educativas que mejoren la retención del conocimiento.

2 Descripción del tema

El sistema Smart History Tutor aborda el dominio de la Historia Universal, ofreciendo respuestas fundamentadas y argumentadas a consultas relacionadas con eventos, procesos, personajes y periodos históricos. La temática seleccionada respondió a la necesidad de desarrollar recursos educativos que brinden información confiable y estructurada para estudiantes en sus procesos de estudio y preparación académica. La solución implementada considera la Historia Universal como un campo extenso y multidimensional, que requiere integrar grandes volúmenes de información procedentes de fuentes como Wikipedia en español, procesarlas mediante técnicas de vectorización semántica y gestionar su recuperación y generación de respuestas de manera eficiente. Además, en su versión extendida, el proyecto implementa un modelo de simulación con agentes estudiantes para optimizar planes de clases, integrando así no solo el aspecto de consulta y respuesta, sino también la planificación didáctica y la evaluación de estrategias pedagógicas orientadas a maximizar el aprendizaje.

3 Soluciones implementadas

3.1 Tutor Inteligente RAG

Para la fase inicial del proyecto, se implementó un tutor inteligente basado en el modelo Retrieve-Augmented Generation (RAG). Este modelo combina técnicas de recuperación semántica con generación de texto, permitiendo que el sistema responda preguntas sobre Historia Universal con un enfoque argumentativo y fundamentado. La arquitectura adoptada fue multiagente, incluyendo los siguientes componentes principales:

- **Crawler Agent:** encargado de la recopilación automatizada de artículos de Wikipedia en español relacionados con Historia Universal, conformando un corpus inicial.
- **Preprocessor Agent:** responsable de la limpieza y procesamiento de los artículos recopilados, realizando la tokenización y segmentación en chunks adecuados para su posterior vectorización.
- **Vectorizer Agent:** generó los embeddings semánticos de cada chunk utilizando modelos de sentence transformers, almacenándolos en índices FAISS para facilitar búsquedas eficientes.
- **Retriever Agent:** recuperó los chunks más relevantes mediante búsquedas semánticas en FAISS, aplicando estrategias de recuperación optimizadas según el tipo de consulta.
- **Generator Agent:** generó respuestas coherentes, contextualizadas y argumentadas a partir de los chunks recuperados, integrando la API de Google Gemini como modelo generativo.

Esta integración de agentes permitió al sistema ofrecer respuestas fundamentadas y superar la limitación de modelos QA tradicionales que extraen solo citas textuales sin elaboración explicativa.

3.2 Simulación con metaheurística y algoritmo genético

Como parte de la ampliación del proyecto, se implementó un módulo de simulación educativa basado en algoritmos genéticos y metaheurísticas. Este módulo tiene como objetivo optimizar planes de clases en función de parámetros pedagógicos definidos, tales como la cobertura temática, el balance entre sesiones de contenido y repaso, y el rendimiento estudiantil simulado.

El proceso se estructuró en las siguientes etapas:

1. **Inicialización de población:** se generaron planes de clases heurísticos aleatorios que cubrían subtemas de Historia Universal, evaluando su cobertura y estructura.
2. **Evaluación de fitness:** cada plan fue evaluado según un fitness que combinaba cobertura temática, balance de sesiones y, posteriormente, las puntuaciones promedio obtenidas por los estudiantes simulados.
3. **Selección, crossover y mutación:** se empleó selección por torneo, cruces entre planes y mutaciones puntuales en subtemas para generar nuevas generaciones de planes, buscando mejorar la calidad de las soluciones.
4. **Iteración evolutiva:** el proceso se repitió por múltiples generaciones, almacenando los mejores planes en términos de fitness y aprendizaje simulado.

De esta forma, se implementó un enfoque metaheurístico evolutivo que optimiza dinámicamente los planes de clases, permitiendo generar distribuciones didácticas más eficientes para la enseñanza de Historia Universal.

3.3 Agentes estudiantes y sistemas difusos

Para evaluar los planes de clases generados, se implementó un modelo de simulación con agentes estudiantes. Cada agente estudiante modeló un perfil de aprendizaje específico, incluyendo parámetros como: tasa base de aprendizaje, probabilidad de omisión, probabilidad de olvido, motivación, estado y ambiente.

El comportamiento de los estudiantes se moduló mediante sistemas difusos, aplicados en dos componentes:

1. Ajuste de aprendizaje y omisión
2. Ajuste de motivación

Durante la simulación, los estudiantes recibieron instrucción según los planes generados, procesaron la información y realizaron tests. Sus respuestas fueron evaluadas comparando la similitud con las respuestas correctas, determinando así el rendimiento de cada plan.

4 Consideraciones de implementación

Durante la implementación del proyecto Smart History Tutor, se tomaron diversas consideraciones técnicas y metodológicas para garantizar su funcionalidad y

escalabilidad. Se seleccionó Python como lenguaje base por su amplia disponibilidad de bibliotecas para procesamiento de lenguaje natural, sistemas difusos, recuperación de información y simulación evolutiva. Para la gestión de la recuperación semántica se empleó FAISS, dada su eficiencia en búsquedas vectoriales de alta dimensión. La generación de respuestas utilizó la API de Google Gemini, elegida por su capacidad de manejar contextos extensos con costos razonables durante la fase experimental. Se implementó un diseño multiagente modular, cada agente con logs independientes para trazabilidad, y se integraron pipelines específicos para la simulación y la optimización genética. Los agentes estudiantes se modelaron con inicialización aleatoria de sus parámetros, permitiendo representar heterogeneidad en sus capacidades de aprendizaje.

5 Justificación teórica-práctica de las decisiones

Cada decisión de diseño e implementación respondió a necesidades prácticas y fundamentos teóricos consolidados. El uso de una arquitectura multiagente alineada con RAG se justificó por su capacidad de integrar módulos especializados para la recuperación y generación de información. La elección de FAISS se sustentó en su eficiencia comprobada para búsquedas de similitud a gran escala. La incorporación de un algoritmo genético respondió a la necesidad de optimizar planes de clases en un espacio de búsqueda grande y no lineal. Los sistemas difusos modelaron variables humanas graduales como motivación y ambiente, ofreciendo un comportamiento más realista. El diseño de agentes estudiantes diferenciados permitió simular un entorno educativo auténtico donde la efectividad de un plan depende de la heterogeneidad estudiantil.

6 Evaluación cuantitativa y cualitativa

Cuantitativamente, el sistema gestionó un corpus de aproximadamente 9000 fragmentos vectorizados. La simulación evolutiva generó planes con coberturas superiores al 80% y puntuaciones promedio estudiantiles entre 0.65 y 0.85. Cualitativamente, el tutor generó respuestas argumentadas y la arquitectura multiagente mostró flexibilidad y escalabilidad. Los modelos difusos aportaron realismo al aprendizaje simulado.

7 Autocrítica: logros, insuficiencias y propuestas de mejora

7.1 Logros

- Tutor RAG multiagente funcional
- Módulo de simulación metaheurística con evaluación estudiantil
- Agentes estudiantes diferenciados con modelos difusos

7.2 Insuficiencias

- Tiempos de vectorización elevados para grandes lotes
- Dependencia de la API Gemini
- Limitaciones en el modelado social y afectivo de los estudiantes
- Corpus limitado a Wikipedia

7.3 Propuestas de mejora

- Usar modelos generativos locales con fine-tuning
- Integrar datasets históricos validados académicamente
- Optimizar la vectorización con técnicas cuantizadas
- Modelar variables afectivas y sociales más complejas

8 Referencias

References

1. Lewis, P. et al.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS (2020). arXiv:2005.11401
2. Jégou, H., Douze, M., Johnson, J.: Billion-scale similarity search with GPUs. arXiv:1702.08734
3. Douze, M. et al.: The FAISS Library. arXiv:2401.08281
4. Gao, Y. et al.: Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997
5. Goldberg, D. E.: Genetic Algorithms in Search, Optimization and Machine Learning. Addison Wesley (1989).
6. Zadeh, L. A.: Fuzzy Sets. Information and Control, vol. 8, pp. 338–353 (1965).
7. Ross, T. J.: Fuzzy Logic with Engineering Applications. Wiley (2016).